

Memoria de Diseño de la Interfaz Web:

Shinydex Pokeapi

Asignatura: Lenguajes de Marcas y Sistemas de Gestión de Información (LMSGI)

Proyecto: Aplicación Web de Consulta y Registro Shiny (PokeAPI)

Curso: 1º de DAM G2

Nombres: Mateo García, Ricardo Crespo y Pablo Soriano

1. Introducción y Temática del Proyecto

Este proyecto consiste en una aplicación interactiva denominada Shinydex Pokeapi. Su propósito principal es permitir a los usuarios consultar información detallada de cualquier Pokémon consumiendo datos en tiempo real de la API pública PokeAPI y, además, ofrecer un diario o registro personalizado donde los entrenadores puedan documentar y guardar sus capturas de Pokémon en su versión alternativa (*Shiny*).

La aplicación combina el consumo asíncrono de datos externos con un sistema de persistencia local para gestionar una colección propia de criaturas capturadas, permitiendo registrar detalles de la obtención como la fecha, los intentos requeridos (*Soft Resets*) y el videojuego en el que se capturó.

2. Estructura General de la Web

El proyecto se compone de tres archivos principales que separan la estructura, la presentación visual y el comportamiento lógico:

- **index.html:** Define el esqueleto del documento, utilizando etiquetas semánticas de HTML5.
- **style.css:** Contiene las reglas de diseño, adaptabilidad (responsividad) y estilos visuales de los componentes.
- **script.js:** Controla el comportamiento dinámico, la comunicación con la API externa, la manipulación de los elementos del DOM y la persistencia de datos.

3. Estructura de la Interfaz (HTML y CSS)

La interfaz se divide de manera intuitiva y limpia en cuatro bloques principales:

A. Cabecera (<header>)

Presenta el título de la aplicación (Shinydex Pokeapi) y una pequeña guía informativa sobre los rangos de IDs de Pokémon válidos para orientar al usuario en sus búsquedas.

B. Sección de Búsqueda y Caja Pokémon (<main>)

Es el núcleo de consulta de la aplicación:

- Control de Búsqueda: Un campo de texto (<input>) donde el usuario introduce el nombre o ID del Pokémon y un botón interactivo (#btnPedir) para realizar la petición.
- Contenedor de Resultados (#cajaPokemon): Caja que se rellena dinámicamente mediante JavaScript. Muestra el sprite del Pokémon, sus datos físicos (altura, peso, tipos), gráficos de barras proporcionales para sus estadísticas base y dos botones adicionales:
 - Un botón para alternar la visualización del sprite entre su forma normal y su variante *Shiny*.
 - El botón temático "¡Lo atrapé!", que activa el proceso de registro en la colección personal.
- Formulario de Registro (#formularioRegistro): Diseñado para permanecer oculto por defecto (display: none;). Se hace visible en cuanto el usuario pulsa en "¡Lo atrapé!". Contiene campos para introducir la fecha de captura, el número de encuentros/resets realizados y un desplegable dinámico para seleccionar el videojuego donde se obtuvo.

C. Listado de Capturas (<section id="seccionShinies">)

Área dedicada a mostrar el progreso de la colección del usuario:

- Un contador en tiempo real que indica la cantidad de Pokémon registrados.
- Un buscador local (#filtroShinies) para buscar capturas específicas dentro de la colección por su nombre.
- Un contenedor flexible (#listaShinies) que organiza visualmente las tarjetas individuales de cada Pokémon shiny registrado mediante un diseño líquido adaptativo (*Flexbox*).

D. Pie de página (<footer>)

Contiene los créditos del proyecto y el enlace hacia el proveedor de datos oficial (PokeAPI).

4. Relación con la Información Almacenada (Persistencia en LocalStorage)

Para conservar los Pokémon registrados por el usuario entre sesiones de navegación, la aplicación utiliza **localStorage**, almacenando las estructuras de datos en formato **JSON** del lado del cliente. Esto evita que la colección del usuario se borre al recargar la página.

A. Modelo de Datos del Pokémon Registrado

Cada captura se representa mediante un objeto de JavaScript estructurado de la siguiente forma:

```
{
  "nombre": "pikachu",
  "imagen":
    "[https://raw.githubusercontent.com/PokeAPI/sprites/master/sprites/pokemon/shiny/25.png](https://raw.githubusercontent.com/PokeAPI/sprites/master/sprites/pokemon/shiny/25.png)",
  "fecha": "2026-05-26",
  "resets": 154,
  "juego": "yellow"
}
```

B. Flujo de Lectura, Escritura y Sincronización

La comunicación entre la interfaz de usuario y los datos almacenados sigue un ciclo cerrado:



1. **Carga Inicial:** Al cargarse la ventana (DOMContentLoaded), el script accede a la clave "misShinies" del almacenamiento del navegador. Si existen datos almacenados, los analiza mediante JSON.parse(); si no, inicializa un array vacío para empezar a trabajar de inmediato:
`let misShinies = JSON.parse(localStorage.getItem("misShinies")) || [];`

2. **Visualización Dinámica:** La función `actualizarLista()` recorre este array utilizando un bucle `for...of`. Para cada Pokémon, crea dinámicamente nodos en el DOM (tarjetas con la imagen del shiny, nombre, fecha, resets y juego de origen) y los introduce en el contenedor `#listaShinies`.
3. **Registro de Nuevos Datos:** Cuando el usuario rellena el formulario y pulsa "Guardar Shiny", el evento `submit` detiene el comportamiento por defecto de la página. El script captura los valores de los inputs, crea el nuevo objeto y lo añade al array con `.push()`. Acto seguido, guarda la lista actualizada convirtiéndola a texto plano con `JSON.stringify()`:
`localStorage.setItem("misShinies", JSON.stringify(misShinies));`
Finalmente, invoca automáticamente a `actualizarLista()` para refrescar la pantalla y ocultar el formulario.
4. **Buscador Integrado:** Al escribir en el buscador de la colección, un evento de escucha (input) filtra el array de datos en memoria utilizando la función `.filter()`, mostrando al instante únicamente las tarjetas cuyos nombres coincidan con la búsqueda.
5. **Eliminación de Registros:** Cada tarjeta cuenta con un botón "Borrar". Al hacer clic en él, se filtra el elemento seleccionado del array en base a su identidad, se sobrescribe el `localStorage` con la nueva lista reducida y se regenera la interfaz visual de forma instantánea.